In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn import metrics
from sklearn.metrics import classification_report, confusion_matrix, roc_curve, auc, accuracy_score, precision_score, recall_score, f1_score, ConfusionMatrixDisplay
from sklearn.preprocessing import label_binarize
```

In [2]:
```
dtree = pd.read_csv('iris.csv')
display(dtree.head())
```

	SepalLength	SepalWidth	PetalLength	PetalWidth	Name
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

In [3]:
```
# Basic Info and Statistics
print("Dataset Info:")
dtree.info()
print("\nDataset Description:")
dtree.describe()
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   SepalLength  150 non-null    float64
 1   SepalWidth   150 non-null    float64
 2   PetalLength  150 non-null    float64
 3   PetalWidth   150 non-null    float64
 4   Name         150 non-null    object 
dtypes: float64(4), object(1)
memory usage: 6.0+ KB

Dataset Description:
```

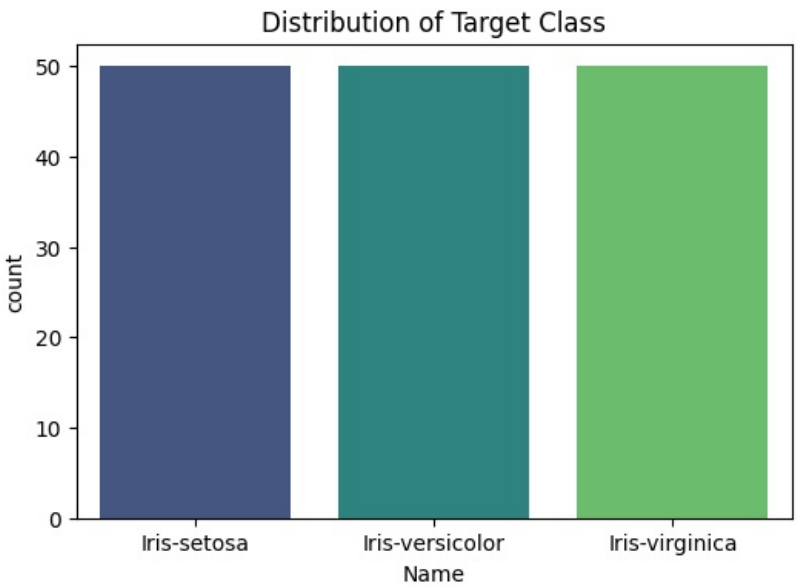
Out[3]:

	SepalLength	SepalWidth	PetalLength	PetalWidth
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

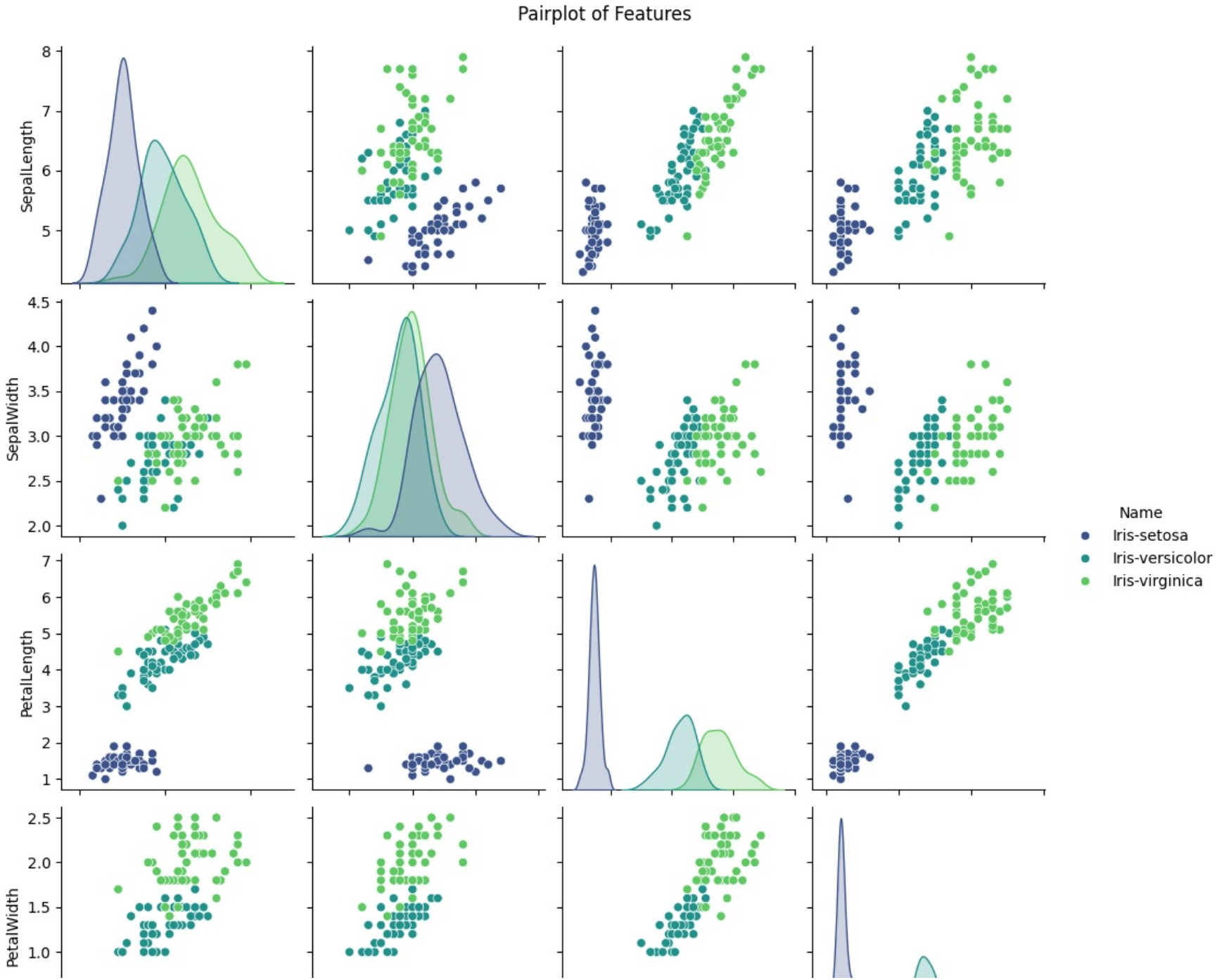
In [4]:
```
# Check for missing values
print("Missing Values:")
print(dtree.isnull().sum())
```

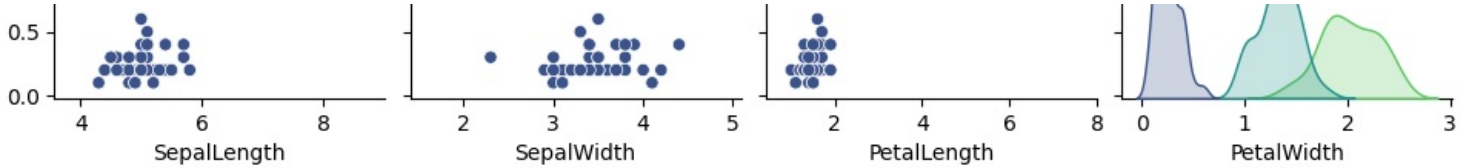
```
Missing Values:
SepalLength    0
SepalWidth     0
PetalLength    0
PetalWidth     0
Name           0
dtype: int64
```

In [5]:
```
# Distribution of Target Class
plt.figure(figsize=(6,4))
sns.countplot(x='Name', data=dtree,hue='Name', palette='viridis')
plt.title('Distribution of Target Class')
plt.show()
```

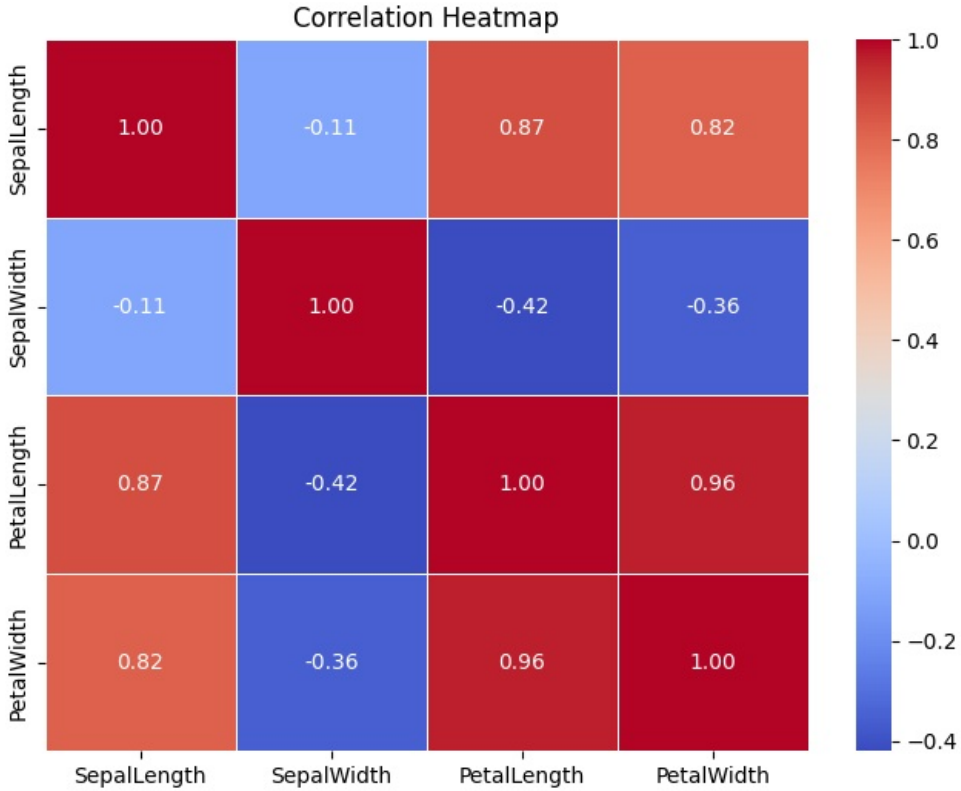


In [6]:
```
# Pairplot for Feature Distributions and Relationships
sns.pairplot(dtree, hue='Name', palette='viridis', diag_kind='kde')
plt.suptitle('Pairplot of Features', y=1.02)
plt.show()
```





In [7]:
```
# Correlation Heatmap
# Select only numeric columns for correlation matrix
numeric_cols = dtree.select_dtypes(include=[np.number]).columns
plt.figure(figsize=(8,6))
sns.heatmap(dtree[numeric_cols].corr(), annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title('Correlation Heatmap')
plt.show()
```



In [8]:
```
X = dtree.drop('Name', axis=1)
y = dtree['Name']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
print(f"Training set shape: {X_train.shape}")
print(f"Testing set shape: {X_test.shape}")
```

```
Training set shape: (120, 4)
Testing set shape: (30, 4)
```

In [9]:
```
# Initialize and train Naive Bayes Model
```

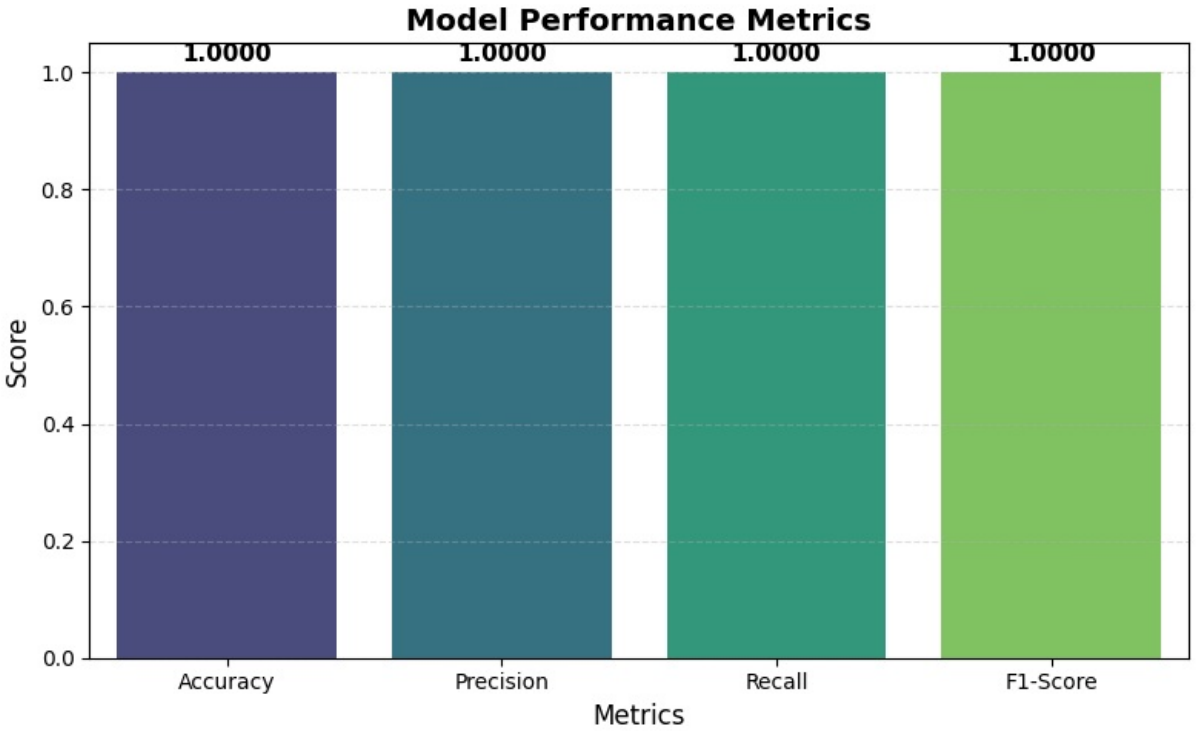
```
clf = GaussianNB()
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
y_prob = clf.predict_proba(X_test) # Probabilities for ROC curve
```

In [10]:
```
# Calculate Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

print("--- Model Performance Metrics ---")
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1-Score : {f1:.4f}\n")

print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
--- Model Performance Metrics ---
Accuracy : 1.0000
Precision: 1.0000
Recall   : 1.0000
F1-Score : 1.0000

Classification Report:
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00         9
 Iris-virginica       1.00      1.00      1.00        11

       accuracy                           1.00        30
      macro avg       1.00      1.00      1.00        30
   weighted avg       1.00      1.00      1.00        30
```

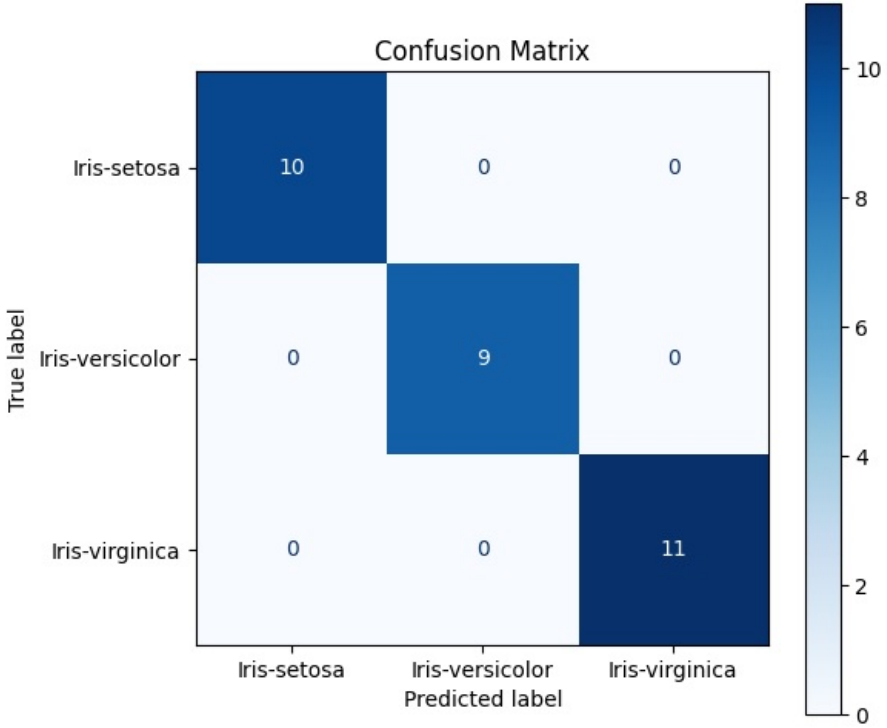
In [11]:
```
# Visualizing Classification Metrics
metrics_dict = {
    'Accuracy': accuracy,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1
}

# Create figure
plt.figure(figsize=(8, 5))

# Bar Plot
ax = sns.barplot(
    x=list(metrics_dict.keys()),
    y=list(metrics_dict.values()),
    hue=list(metrics_dict.keys()),
    palette='viridis',
    legend=False
)
```

```
# Axis limits and labels
plt.ylim(0, 1.05)
plt.xlabel("Metrics", fontsize=12)
plt.ylabel("Score", fontsize=12)
plt.title('Model Performance Metrics', fontsize=14, fontweight='bold')

# Add value labels on bars
for i, v in enumerate(metrics_dict.values()):
    ax.text(
        i,
        v + 0.02,
        f"{v:.4f}",
        ha='center',
        fontsize=11,
        fontweight='bold'
    )

# Add grid for better readability
plt.grid(axis='y', linestyle='--', alpha=0.4)

# Improve layout
plt.tight_layout()

# Show plot
plt.show()
```

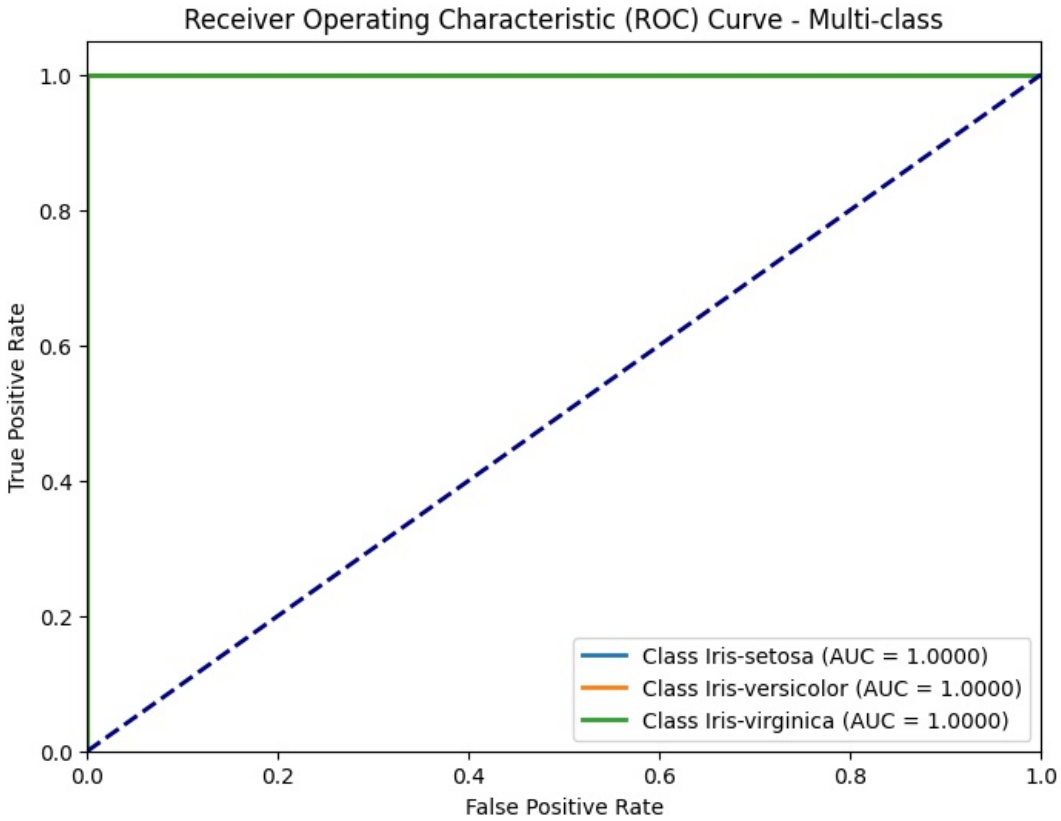


In [12]:
```
# Confusion Matrix Visualization
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=clf.classes_)
```

```
fig, ax = plt.subplots(figsize=(6, 6))
disp.plot(cmap='Blues', ax=ax, values_format='d')
plt.title('Confusion Matrix')
plt.show()
```



In [13]:
```
# ROC Curve and AUC (Multi-class)
y_test_bin = label_binarize(y_test, classes=clf.classes_)
n_classes = y_test_bin.shape[1]

plt.figure(figsize=(8, 6))
for i in range(n_classes):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_prob[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, lw=2, label=f'Class {clf.classes_[i]} (AUC = {roc_auc:.4f})')

plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - Multi-class')
plt.legend(loc="lower right")
plt.show()
```



In [14]:
```python
# Example prediction on a new data point
import pandas as pd

# Define the data (SepalLength, SepalWidth, PetalLength, PetalWidth)
sample_data = [[5.1, 3.5, 1.4, 0.2]]

# Convert to DataFrame with the same column names used during training
sample_df = pd.DataFrame(sample_data, columns=X.columns)

prediction = clf.predict(sample_df)
prob = clf.predict_proba(sample_df)

print(f"Sample Data: {sample_data[0]}")
print(f"Predicted Class: {prediction[0]}")
for i, cls in enumerate(clf.classes_):
    print(f"Probability for {cls}: {prob[0][i]:.4f}")
```

```
Sample Data: [5.1, 3.5, 1.4, 0.2]
Predicted Class: Iris-setosa
Probability for Iris-setosa: 1.0000
Probability for Iris-versicolor: 0.0000
Probability for Iris-virginica: 0.0000
```